Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Kontrol Akışı

- Kontrol akışı (execution sequence), bir programın hangi sırayla çalıştığını ifade eder.
- Bu akış farklı seviyelerde incelenebilir:
  - İfade düzeyi (expression level)
    - Operatör önceliği (precedence) ve bağlaşım (associativity).
  - Deyimler arası (statement level)
    - if, loop gibi yapılar yürütme sırasını değiştirir.
  - Program birimleri arası (unit level)
    - Fonksiyon ve prosedür çağrıları kontrol akışını belirler.

Imperatif Dillerde Hesaplama

- Imperatif programlama dillerinde hesaplamada
  - İfadeler değerlendirilir
  - Sonuçlar değişkenlere atanır
  - Örnek:
    - x = y + 2;
  - Ancak:
    - ❗ Sadece atama ile yazılmış programlar **yetersizdir**
  - Neden?
    - Karar veremez
    - Tekrar yapamaz

Kontrol Yapılarının Gerekliliği

- Bir programın güçlü olması için 2 temel mekanizma gerekir:
  - 1. Seçim (Selection):
    - Alternatif yollar arasında seçim yapar.
  - 2. Tekrar (Iteration):
    - Aynı işlemi tekrar ettirir.
  - Bu işlemleri sağlayan yapılar:
    - 👉 Kontrol deyimleri (control statements)

Fonksiyonel Dillerde Kontrol

- Fonksiyonel dillerde:
  - Hesaplama → fonksiyonlar ve ifadeler ile yapılır.
  - Kontrol akışı → yine ifadeler ve fonksiyonlarla sağlanır.
  - Önemli fark:
    - Imperatif → kontrol deyimi kullanır.
    - Fonksiyonel → kontrol ifadesi kullanır
    - 👉 Ancak mantık benzerdir

Kontrol Yapıları

- Gerçek dünyada önemli olan:
  - ✓ Yazılabilirlik (writability)
    - Kodun kolay yazılması
  - ✓ Okunabilirlik (readability)
    - Kodun kolay anlaşılması
  - Denge problemi:
    - Çok az kontrol yapısı → zor yazım.
    - Çok fazla kontrol yapısı → zor okuma

Kontrol Yapıları

- Kontrol yapısı:
  - ➡️ Bir kontrol deyimi + kontrol ettiği kod bloğu
  - Örnek:

```
if (x > 0) {
    y = x;
}
```

- Çoklu Giriş Problemi:
  - Bir kontrol yapısına birden fazla giriş noktası olmalı mı?
  - Cevap:
    ❌ Hayır (modern yaklaşım)
  - Neden?
    - Karmaşıklık artar
    - Okunabilirlik düşer
  - Not:
    - Sadece goto varsa mümkün

Kontrol Yapıları

- Çoklu Çıkış (Exit) Konusu
- Soru: Bir kontrol yapısından birden fazla çıkış olmalı mı?
- Cevap:
  ✓ Evet (sınırlı şekilde)
- Sebep:
  - Doğal akış bozulmazsa sorun yok.

- Tehlikeli Durum
- Eğer çıkış herhangi bir yere gidebiliyorsa :
  - goto gibi davranır.
  - okunabilirlik düşer.
  - hata riski artar.

Kontrol Yapıları - Seçim Deyimleri

- Programda birden fazla yürütme yolu arasından seçim yapmayı sağlar.
- Önemi: Tüm programlama dillerinde temel yapıdır.
- Algoritmaların vazgeçilmez parçasıdır.
- Teorik temel:
- ➡️ Corrado Böhm
- ➡️ Giuseppe Jacopini
- Sonuç:Seçim + döngü = tüm algoritmalar
- Türler:
  - İkili seçim (two-way)
  - Çoklu seçim (n-way)

- İkili Seçim

```
if control_expression
    then clause
else
    else clause
```

- control_expression → karar verir.
- then → doğruysa çalışır.
- else → yanlışsa çalışır.

```
if (x > 0)
    y = x;
else
    y = -x;
```

Kontrol Yapıları - Seçim Deyimleri

- İkili seçim tasarımında 3 temel soru:
  - **1. Kontrol ifadesi nasıl olmalı?**
    - Boolean mı?
    - Sayısal ifade olabilir mi?
  - **2. then / else nasıl tanımlanır?**
    - Tek satır mı?
    - Blok mu?
  - **3. İç içe (nested) yapıların anlamı nasıl belirlenir?**
- Bu sorular dil tasarımını doğrudan etkiler.

- Kontrol İfadesi Türleri
  - C (C89): Boolean yok, sayısal ifade

```c
if (x)   // 0 = false, diğerleri true
```

  - Java: Boolean zorumlu

```java
if (x > 0)   // boolean zorunlu
```

  - Python: Hem boolean hem sayısal ifade

```python
if x > 0:
    print(x)
```

Kontrol Yapıları - Blok Yapısı

- Farklı dillerde seçim ifadesinin altın yer alan kısım aşağıdaki şekilde blok olarak tanımlanır.

Dil	Blok Yapısı
C	`{}`
Python	girintileme
Ruby	`end`

Python

```python
if x > y:
    x = y
    print("case 1")
```

Aynı girintideki tüm satırlar blok içindedir: → then başlangıcıdır. Parantez yoktur

Nested if ve Dangling Else Problemi

Java

```java
if (sum == 0)
    if (count == 0)
        result = 0;
    else
        result = 1;
```

Ruby

```ruby
if sum == 0 then
    if count == 0 then
        result = 0
    else
        result = 1
    end
end
```

Ruby'de end kullanımı nedeniyle belirsizlik oluşmaz.

- Problem else hangi if'e ait.
- Java da else → en yakın if'e bağlanır.
- Çözüm blok için parantez kullanımı.

Java

```java
if (sum == 0) {
    if (count == 0)
        result = 0;
} else {
    result = 1;
}
```

Nested if ve Dangling Else Problemi

```python
if sum == 0:
    if count == 0:
        result = 0
else:
    result = 1
```

Python'da girintiye göre anlam değişir:
else aynı hizadaysa → dış if'e bağlı
içeri kaydırılırsa → iç if'e bağlı

- Problem else hangi if'e ait.
- Java da else → en yakın if'e bağlanır.
- Çözüm blok için parantez kullanımı.

```java
if (sum == 0) {
    if (count == 0)
        result = 0;
} else {
    result = 1;
}
```

Fonksionel Dillerde if

- Fonksiyonel dillerde if bir **ifade (expression)**'dir.
- Burada ifade demekten kasıt bir değer döndürür anlamındadır.
- if değer üretir
  - y değişkenine atanır.
  - else zorunludur (değer döndürmek için).

```fsharp
let y =
    if x > 0 then x
    else 2 * x
```

Çoklu seçim ifadeleri

- Programın birden fazla olası yürütme yolundan birini seçmesini sağlar.
- İki yönlü seçimlerin (if-else) genelleştirilmiş halidir.
- Temel fikir:
  - Bir değişkenin veya ifadenin değerine göre farklı kod blokları çalıştırılır.

```c
if (x == 1) {...}
else if (x == 2) {...}
else if (x == 3) {...}
```

Bu yapı büyüdükçe: Okunabilirlik azalır. Kod karmaşık hale gelir. Bu yüzden **switch / case gibi özel yapılar geliştirilmiştir.**

Çoklu seçim ifadeleri

- **Neden Gerekli?**
- Çoklu seçim ihtiyacı:
  - Menü sistemleri
  - Durum makineleri
  - Komut yorumlayıcılar
- İki-way selector ile çözüm:
  - Çok fazla **nested if**
  - **karmaşık ve hataya açık.**
- goto ile çözüm:
  - **okunabilirliği ciddi bozar.**
  - yapı kontrolünü zorlaştırır.

- Bir multiple selector tasarlanırken şu sorular önemlidir:
- Kontrol ifadesi tipi nedir?int, char, string, boolean?
- Seçilen bloklar nasıl yazılır?
  - tek satır mı?blok mu?statement sequence mi?
- Tek bir blok mu çalışmalı?yoksa birden fazla blok çalışabilir mi?
- Case değerleri nasıl belirtilir?
- Hiçbir case eşleşmezse ne olur?
  - hata mı?
  - default mu?
  - hiçbir şey mi yapılmaz?

Çoklu seçim ifadeleri

- C dilinde
  - expression → kontrol ifadesi
  - case → olası değerler
  - default → hiçbir case eşleşmezse çalışır
  - ✓ Özellikler:
    - integer, char gibi **discrete type** kullanılır
    - case değerleri sabit olmalıdır.
    - case blokları otomatik olarak sonlanmaz.

```c
switch (expression) {
  case 1: statement;
  case 2: statement;
  default: statement;
}
```

```c
switch (index) {
  case 1:
  case 3:
    odd++;
  case 2:
  case 4:
    even++;
}
```

1 veya 3 geldiğinde: hem odd++ hem even++ çalışır.
✓ Buna fall-through denir.

Çoklu seçim ifadeleri

- Fall-through'u önlemek için break kullanılır. Break switch içerisinden hemen çıkmayı sağlar. Break aslında **kısıtlanmış goto** gibi davranır.

```c
switch (index) {
  case 1:
  case 3:
    odd++;
    break;

  case 2:
  case 4:
    even++;
    break;
}
```

```c
switch (expression) {
  case 1: statement;
  case 2: statement;
  default: statement;
}
```

```c
switch (index) {
  case 1:
  case 3:
    odd++;
  case 2:
  case 4:
    even++;
}
```

1 veya 3 geldiğinde: hem odd++ hem even++ çalışır.
✓ Buna fall-through denir.

Çoklu seçim ifadeleri

Ruby

```ruby
case
when x > 10 then puts "big"
when x > 5 then puts "medium"
else puts "small"
end
```

Python

```python
if count < 10:
    bag1 = True
elif count < 100:
    bag2 = True
elif count < 1000:
    bag3 = True
```

Çoklu seçim ifadeleri

- Multiple selection nasıl çalışır?
  - 1. Basit yöntem:

    if t == case1 goto L1

    if t == case2 goto L2

    ❌ Yavaş:O(n) karşılaştırma

  - 2. Linear search:tablo + döngü
  - 3. Hash table: hızlı (~O(1))
    - ❌ range desteklemez.
- 4. Binary search: orta hız (~O(log n))
- 5. Jump table (array): en hızlı yöntem.
  - labels[value] → direkt atlama
- 📌 Compiler:case sayısına göre yöntemi seçer.